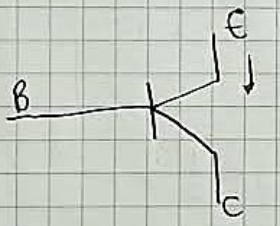
# APUNTES

## Sistemas Digitales — Clase 1 - 30/09

- Microcontroladores.- computadora que cumple una sola función.
- Sistema embebido.- es un sistema que puede ejecutar una sola tarea en tiempo real.

| Microcontrolador | Microprocesador |
|---|---|
| tiene CPU, memoria y periféricos en un chip | solo tiene CPU |
| Consumo Bajo | Consumo Medio |
| Control dedicado embebido | Propósito general |




para utilizar un transistor hay que revisar la dataset para ver que voltaje hace que arroge un 0 lógico o 1 lógico

si 5V (4V 5V) o 0V (3V 2V)

- Los microcontroladores trabajan con 32 bits máximo dentro de la PC para registro de memoria
- Los microcontroladores ya tienen incluidos tarjeta
- En microcontroladores no se borran los datos.
- Memoria es la que almacena los datos de manera temporal o permanente, es intermediaria entre CPU y disco
- Bus datos.- una autopista bidireccional de datos de la memoria al procesador
- Volátiles: Pierden info cuando no tienen energía (RAM)
- No volátil: Guarda información sin importar que. Realiza lectura y escritura.

# Sistemas Digitales

8 bits = 1 byte

Hexadecimal = para páginas web

Binario = comunicación con la maquina

Octar = se trabaja en Linux

* Función del microprocesador ser la cabeza de un dispositivo
* Buses.- Van y vienen, transportan, llevan control datos y la direcciones, ancho del bus
* En sambleador.- Lenguaje de bajo nivel

¿Que es un sistema Embido?

Se dedica a una tarea fija

- Hace una sola cosa, siempre lo mismo.
- Recursos limitados
- Función invisible

(1 Flip flop guarda 1 bit)

* Registros.- castilleros internos rapidos de memoria

SLOW PROGRESS IS STILL PROGRESS

* Unidad de control.- Jefe que da instrucciones y las procesa
* Contador Esta lista para pasar instrucciones
* Nucleo CPU completa, trabajo paralelo

Ritmo de Reloj (Reloj) (ESP 32 tiene 2 nucleos)

- Buscar .- busca linea
- Decodificar.- entiende la linea
- Ejecutar.- se ejecuta linea




Buses de datos es de (8 bits)

Dentro del CPU se van ejecutando las funciones según el registro de entrada.

# Sistemas Digitales

Bus : Direcciones, Datos, Control

Von Neuman: un solo bus de datos (todo se utiliza 1 vez)

Harvard.. Tiene 2 buses

Tres Memorias

* Flash .- nose borra, almacena
* SRAM .- se borra, solo guarda mientras esta encendido y solo guarda variables despues de 3os operacio.
* EPROM .- no se borra pero es muy delicado, se guarda los datos, tiene ciclos se prende y puede colver con lo que tenia

1 byte = 8 bits  1KB = 1624 byte  1MB = 1024 KB

emulo .- como que imita algo

los bits son energia electrica

Mega = $10^6$

Periféricos

ESP32




Bajo 0  Alto 1

Hexadecimal a binario

00 10  00 00

↓2  ↓0

YOU ARE MAGIC

# Compuertas

AND: 1 solo si ambos son 1

OR: 1 si alguno es 1

XOR: 1 si son distintos

NOT: inviente

| | | AND | OR | NOT | XOR |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 | 0 | 1 |
| 1 | 1 | 1 | 0 | 1 | 0 |

La mascara 1 << n

1 << 5 = 0x 20

es igual

(^) → XOR

(&) → AND

(|) → OR

Simulación Sistemas Digitales

Reg 1 = (1 << 0)

```
  1 0 1 0 1 0 1 1
+ 0 0 0 0 0 0 0 1
-----------------
  1 0 1 0 1 0 1 1
```

Ensamblador

Herramientas del microcontrolador

RISC (poca funcionalidad) CISC (bastantes)

LDI.- Cargar inmediatamente

ADD.- Sumar registros

SBI.- escribe un 1

CBI.- escribe un 0

RJMP.- salto de linea

* El codigo que ingreso en lenguaje program con el compilador se transforma ensamblador luego se transforma a binario para que entienda la maquina, posterior se activa los resgistros segun el mensaje binario y al final salen los pulsos de reloj en base al mensaje binario y los registros ocupados.

Analisis.

Metodo Newton - Raphson

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

SLOW PROGRESS IS STILL PROGRESS

# Sistemas Digitales

DDRB.- como se comporta el puerto

↳ es 1 es salida → es igual a 5V
entrada ↳ 0 → 0v

PORT.- indica como actua

Sensores analogicos

Para leer el pin del controlador {PINB}

1. 0x81 → 10000001
2. PORTB &= ~(1<<4)
3. PIND2 lee un 0 logico

0 y 1 → Activa el pullup

DDRB = 0 y PORTP = 0 → alta impedancia

# Sistemas

* Registro → memoria y registro
* Periferico → circuito interno que cumple una funcion especifica para liberar carga de la CPU
* PIN → conexion fisica hacia el mundo exterior con el interior

* GPIO Entradas y salidas de poposito general

Comunicación Bloch hasta la CPU 6 11 xq es aparte

GPIO

in → Copia

out → escribe

W1TS → Setea

Bits 1 → enciende

0 → no se toca

eor calculo con la copia

W1TC → Limpia

Bits en 1 → se apaga

0 → no se toca

- Si una lectura se interrumpe al pasar a la salida se obtendra un error

Sistemas Digitales

esp trabaja con 32

* Temporizadores es un circuito dentro del microcontrolador es un pereferico que se puede sacar un tiempo cuenta pulsos
* delay .- no hace nada hasta que se acabe el tiempo
* millis .- todo sigue trabajando pero hay que revisar

cada vez pregunta si ya paso el tiempo

* Temporizador: alarma exacta de que se acabo el tiempo
* contador: registro del temporizador (almacena los bits)

CPU arranca el temporizador

* Reloj: señal que sube y baja a ritmo fijo
* APB: Bus de periferico CPU se comunica con los perifericos del circuito de 80MHz (125 nano)
* Alarma: numero que se guarda en un registro cuando el contador lo alcanza genera un aviso

formula = alarma = tiempo x frecuencia

* Resolucion: Duracion de cada tick

Ejemplo:

10 000 ticks la alarma suena en los 2500 ticks

calcular la resolucion

↳ 1 / 10000 = 0,00012 = 1µs

t. alarma → = 2500 x 10 000 = 0,25

Prescaler → 80 000 000 / 10 000 =

* Autorecarga: el contador vuelve a 0 cuando se cumple la alarma
* Desbordamiento: capacidad que tiene (no alcanza la alarma tiene que repetir)
* API: escribe los registros
* ISR: es la rutina de servicio de interrupción

Si [redacted] → Sistemas Digitales

* Sondeo (polling) .- preguntar una y otra vez en el loop() si algo ya pasó

* Flanco.- Es un instante en el que una señal cambia de 0 a 1 o 1 a 0

* Interrupción = algo se le avisa
  ↳ interna = alarma ↳ externa = con algo de por fuera

* ISR .- función que se ejecuta al llegar el aviso lo llama el Hadware → interrupción

* SRAM.- mesa de trabajo al instante {DRAM = datos} {IRAM = instrucc}

* watchdog .- Perro guardian mtls

* GPIO .- entrada y salidas con propósito general

- attachInterrupt()

- ISR Bolan() .- función que dice que va a trabajar con interrupciones
  ↳ no va dentro de delay() etc.

- toda la ISR lleva (IRAM_ATTR) siempre

- el cambiador no sabe que existe la ISR

* volatile .- esta variable cambia a las espaldas y hay que revisar desde la RAM